ProofRank

Intelligent Candidate Discovery & Ranking

Senior AI Engineer (Founding Team) - Redrob AI | Team Caraxes

1. Solution Overview

What is the proposed solution?

ProofRank is a CPU-only hybrid ranker that scores 100,000 candidate profiles against a single fixed job description (Senior AI Engineer, Founding Team) and returns a trustworthy top-100 shortlist. It combines dense semantic retrieval (FAISS over sentence-transformer embeddings) with sparse lexical retrieval (BM25), fuses them with Reciprocal Rank Fusion, then re-scores survivors with a career-proof-first composite, applies a behavioral availability multiplier, hard-drops adversarial honeypot profiles, and emits a ranked CSV with grounded, rule-based reasoning for every candidate.

What differentiates it from traditional candidate matching?

- Career proof over keywords: the dominant signal (weight 0.38) is evidence in the candidate's career_history, not their self-declared skills list. This directly defeats the keyword-stuffing trap built into the dataset.
- Adversarial-aware by design: layered honeypot, trap-title, and keyword-stuffer detectors remove 16,130 manipulated profiles before ranking - most matchers have none.
- Behavioral availability as a multiplier: a perfect-on-paper but inactive or unresponsive candidate is down-weighted, not surfaced.
- Closed-loop evaluation: a silver-label NDCG/MRR/MAP harness measures every change, so design decisions are validated, not guessed.
- Zero-hallucination explanations: reasoning is composed from real profile fields with no LLM in the ranking path.

Key requirements extracted from the JD

JD signal	How it is encoded
Embeddings retrieval (must-have)	Dense FAISS + retrieval evidence family
Vector DB / hybrid search (must-have)	FAISS/BM25 terms, hybrid RRF
Eval frameworks: NDCG/MRR/MAP (must-have)	Production+eval evidence family
5-9 years experience	YoE fit scoring (soft band)
Pune / Noida / India location	Location tier + boost
Product company, scrappy shipper	Product-company + shipper signals
Disqualifiers (research/consulting/	Anti-pattern penalties +
LangChain/title-chaser/CV-only)	honeypot hard-drop

2. JD Understanding & Candidate Evaluation

Which candidate signals matter most?

Signals are combined as an interpretable weighted composite. The priority order (by weight) reflects the JD's own emphasis on proven systems work:

Signal	Weight	Rationale
Career evidence	0.38	Proven retrieval/ranking/production work
Retrieval RRF	0.14	Semantic + lexical relevance to JD
Title tier	0.13	Strong ML/AI/search titles vs trap titles
Anti-pattern penalty	-0.12	Stuffer/consulting/research/title-chase
YoE + location fit	0.09	5-9 band, Pune/Noida/India
Skill trust	0.07	Endorsement x proficiency x duration
Assessment / product / edu	0.05-0.04	Supporting quality signals
Behavioral multiplier	x0.70-1.15	Availability modifier (23 signals)

How is fit evaluated beyond keyword matching?

- Career-narrative evidence families (retrieval, ranking, production, LLM, plain-language) are scored from career_history, weighted 0.82 career-text vs 0.18 summary - so claims must appear in real work history.
- Title-vs-substance gap: the JD's explicit Tier-5 case (a 'Data Scientist' who actually built a production recommendation system) is surfaced by scoring the strongest career role, not just the current title.
- Skill trust discounts keywords by endorsements, proficiency, and duration, so a stuffed skills list scores poorly.
- 19 of the 23 Redrob behavioral signals are consumed as an availability modifier.

3. Ranking Methodology

How does the system retrieve, score, and rank?

- Retrieve: four ranked lists - FAISS career + FAISS full (MiniLM 384-dim) + BM25 full + BM25 career - fused via Reciprocal Rank Fusion (rrf_k=60, top_k=3000), plus a safety pool of high-evidence candidates.
- Score: weighted linear composite of 11 features minus anti-pattern penalty plus a rank-time bonus, clamped to [0,1].
- Modify: behavioral multiplier (recency, response rate, notice period, open-to-work, etc.) bounded to [0.70, 1.15].
- Rank: honeypot hard-drop, top-10 quality guard (min career evidence 0.55, min title tier 0.65), availability guard for ranks <=25, fill, then strictly monotonic scores.

Models, algorithms, and heuristics used

- Models: sentence-transformers all-MiniLM-L6-v2 (384-dim), FAISS (inner-product), Okapi BM25.
- Algorithms: Reciprocal Rank Fusion; interpretable weighted-linear scoring; multiplicative behavioral modifier.
- Heuristics: rule-based honeypot / trap-title / keyword-stuffer / consulting-only / research-only / LangChain-only / title-chaser detectors.

How are multiple signals combined?

composite = sum(weight_i x feature_i) - anti_pattern_penalty + rank_time_bonus; final = composite x behavioral_multiplier. Guardrails then enforce quality and availability at the top of the list. The design choice is interpretable scoring validated by an offline NDCG harness, rather than an opaque learned model - appropriate for a CPU-only, fully reproducible, explainable submission.

4. Explanations, Trust, and Data Quality

How are ranking decisions explained?

Every candidate receives an evidence-led, rank-aware justification that leads with the signal that actually drove the rank (career proof, title-vs-substance gap, evaluation rigor, location, or product-company background), folds concerns into prose, and varies phrasing deterministically by candidate.

How are hallucinations / unsupported justifications prevented?

- No LLM in the ranking path: reasoning is pure string composition from real fields.
- Every clause maps to a concrete profile field (career terms, YoE, location, response rate); nothing is generated that is not in the data.
- Deterministic and reproducible: phrasing variants are chosen by a hash of candidate_id, so identical inputs always produce identical output.

How are inconsistent, low-quality, or suspicious profiles handled?

- Honeypot detector (timeline contradictions, expert skills with zero duration, zero-endorsement skill stuffing, education/experience conflicts) -> hard drop (16,130 removed).
- Trap titles, keyword stuffers, consulting-only, research-only, LangChain-only, and CV/speech-without-IR -> anti-pattern penalties.
- Near-duplicate career narratives -> tie-break penalty.
- Result: the submitted top-100 contains 0 honeypots, 0 trap titles, 0 stuffers, 0 consulting-only profiles.

5. Complete Workflow (JD input -> ranked output)

Offline (pre-computation, ~85 min, declared): 100K candidates -> career/full narratives -> MiniLM embeddings -> dual FAISS indexes; BM25 (career + full); rule-based structured features -> features.parquet; pre-embedded JD vector.

Online (CPU, no network, ~15 seconds): JD query -> hybrid RRF retrieval (top 3000) -> honeypot hard-drop -> career-proof composite score -> behavioral multiplier -> top-10 and availability guards -> top-100 -> monotonic scores -> evidence-led reasoning -> submission.csv -> format validation -> honeypot audit -> NDCG eval.

6. Results, Insights, and Constraints

What demonstrates ranking quality?

Metric (offline silver-label harness)	Value
NDCG@10	0.9552
NDCG@100	0.8716
MRR (relevance >= 2)	1.0000
MAP@100 (relevance >= 2)	0.9667
Top-10 relevance histogram (3/2/1/0)	9 / 1 / 0 / 0

Metric (offline silver-label harness)	Value
Top-100 disqualified candidates	0
Top-100 in 5-9 year band	78%
Top-100 at product companies	97%

Note: labels are weak supervision (a transparent JD rubric), not ground truth; absolute values are inflated, so they are used for relative A/B comparison of pipeline variants - exactly what an offline benchmark is for. A reweighting experiment was tested, regressed NDCG@100 from 0.8716 to 0.8560, and was reverted: eval-driven discipline, not guesswork.

Full-pool proof (results truly come from 100K)

- features.parquet contains 100,000 candidates; 16,130 honeypots excluded; 83,870 scored.
- All 100 submitted candidates fall within the top 119 of the full 83,870-candidate ranking; 94 of 100 are in the full top 100. submission.csv is the genuine head of a full-pool ranking.
- Dataset insight: only ~125 candidates carry strong career proof for this JD, confirming the JD's own statement that few true matches exist.

Runtime and compute constraints

- CPU-only; no GPU used for inference.
- No network during ranking: the JD vector is pre-embedded and cached.
- End-to-end ranking ~15 seconds, far under the 5-minute / 16 GB budget.
- Pre-computation (~85 min) is declared and permitted; reproduce_command runs rank.py against the cached indices.

7. Verdict: Is the architecture worth it?

Yes. The architecture is strongest on exactly the dimensions this challenge weights most: JD-intent comprehension, career-proof-over-keywords, adversarial honeypot defense, explainability with zero hallucination, and full reproducibility within the compute limits. The one deliberate limitation is that scoring is an interpretable, validated heuristic rather than a trained learning-to-rank model - a sound trade-off for a CPU-only, explainable, reproducible system, with the evaluation harness already in place to train and validate an LTR model as the natural next step.